

Argo & Atlas — the guide

Every command and instruction for the two apps, in one place. Argo is Thomas's pocket-money app; Atlas is Joe's stats page. Both run entirely on GitHub: an Action syncs from Garmin every hour and publishes a page; the data is JSON in the repository. Generated 21 September 2026 from the code as it stood.

The addresses

	Argo (Thomas)	Atlas (Joe)
His page	https://lonewonderer1976.github.io/argo/	https://lonewonderer1976.github.io/atlas/
The repository	https://github.com/LoneWonderer1976/argo	https://github.com/LoneWonderer1976/atlas
Actions (the buttons)	.../argo/actions	.../atlas/actions
Statements (issues)	.../argo/issues	—
On the PC	C:\Argo	C:\Atlas

Both pages install on a phone from the browser's **Add to Home Screen**. Both are public pages (GitHub Pages on the free plan), as agreed.

One-time setup

Argo is done. Atlas has one step left, which needs Joe's Garmin email and password typed by you (nothing stores them; the tokens they produce last about a year):

```
cd C:\Atlas
python -m atlas.login
```

That stores the Garmin tokens as the repository secret GARMINTOKENS, starts the first sync on GitHub, and fetches his activities to the PC as well. The page fills a few minutes later.

If either app's *sync* Action ever starts failing to log in (about a year on), run the same login command in that app's folder again — `python -m argo.login` for Argo.

Everything else was scripted: the repositories, GitHub Pages, the page addresses. To rebuild any of it (for a new copy of either app, say), `python setup.py` in the folder is safe to re-run and skips what is already done. It needs the GitHub CLI logged in: `gh auth login --web`, once.

Argo — the week

The week runs **Monday to Sunday**, UK time. The ledger pays from **1 May 2026**; the Easter eggs and the steps count from **21 September 2026**. Everything is derived from the stored activities on every run, so a rule change re-scores every week.

What it pays (the defaults — every one is a setting, see [Changing the rules](#))

	points	money at 25p a point
Running	4 per mile + 1 per 25 m climbed	£1.00 a mile + 1p per metre climbed
Walking	2 per mile + 1 per 50 m climbed	50p a mile + ½p per metre climbed
Cycling	1 per mile + 1 per 100 m climbed	25p a mile + ¼p per metre climbed
Kayaking	4 per mile	£1.00 a mile
Swimming	1 per 100 m	25p per 100 m
Steps	2 per 10,000, gross, every day	50p per 10,000
Anything else (football, breathwork...)	0 — shown, flagged, unpaid	—

Money is rounded on the week's total, not per activity. A week is **current** (still earning), **owed** (finished, unpaid), **paid**, or **empty** (finished, nothing earned).

The Sunday statement and paying him

Every Sunday at about 21:45 an **issue** opens on the Argo repository titled with the week and its money; GitHub emails it to you. It lists every activity with its points, anything to look at (implausible speed, no heart rate, an unscored sport), the steps, the eggs found, and the running total owed.

Pay Thomas, then reply to that email (or comment on the issue) with one line:

reply	what happens
paid	that week is marked paid today; the issue closes; his page moves it from owed to paid
paid all	every unpaid week up to and including that one is marked paid (for a backlog)
paid cash	the same as paid, with the word kept as a note
strike 12345678 the car	that activity earns nothing, with the reason shown on his page (ids are in the statement's table)
unstrike 12345678	puts it back
sport 12345678 kayak	the watch said "other"; this is what it was (run, walk, cycle, swim, kayak)
set steps_pts_per_10k 3	changes a setting (the list is under Changing the rules)
reset week_cap_points	puts a setting back to its default; reset all for every one

The bot answers on the issue within a minute with the new figures. Only your replies are listened to. To get a statement without waiting for Sunday: [Actions](#) → *statement* → *Run workflow*, with a week's Monday (blank = the last completed week).

The *pay* workflow ([Actions](#) → *pay* → *Run workflow*) does the same things from a form, if you prefer that to replying.

The Easter eggs

118 hidden milestones, each with its own message — first mile, two-mile run, ten-mile ride, Ben Nevis's height climbed, £100 ("the Golden Fleece"), a ten-week streak, a 20,000-step day, a million steps... The Trophies panel does not exist on his page until the first one is won; then each new one gets a full-screen fanfare when he next opens the page, and a cabinet lists the ones he has. Only won eggs leave the server, so nothing on the page spoils the rest. The list is `C:\Argo\argo\milestones.py`.

Argo — changing the rules (the admin panel)

Every number is a **setting** with a default in the code and an override kept in `data/settings.json`. Three ways to change one — the same table behind all three:

1. The form, from a phone or PC

GitHub → the argo repository → **Actions** → **settings** → **Run workflow**. Fill in only what you want to change; blank fields stay as they are. *other* takes anything not in the named fields as key=value pairs (e.g. `eggs_start=2026-10-01 run_ascent_per_100m=4`); *reset* takes one setting's name, or all. A bad value is refused with the reason and nothing is changed. Every change is a commit.

2. A reply on any statement issue

```
set steps_pts_per_10k 3
reset week_cap_points
```

3. On the PC

```
cd C:\Argo
python -m argo.settings # every setting: current, default, overridden
python -m argo.settings --set pence_per_point 30 --apply
python -m argo.settings --set week_cap_points 40 --set eggs_start 2026-10-01 --apply
python -m argo.settings --reset all --apply
```

The settings

setting	default	what it is
pence_per_point	25	pence paid per point
week_cap_points	none	most points paid in a week; none = no cap
scheme_start	2026-05-01	the day the ledger opens
eggs_start	2026-09-21	the day the Easter eggs start counting
steps_start	2026-09-21	the day steps start counting
steps_pts_per_10k	2	points per 10,000 steps
steps_per_mile_deducted	0	steps not paid per recorded mile on foot (0 = gross)
run_per_mile / walk_per_mile	4 / 2	points per mile
cycle_per_mile / kayak_per_mile	1 / 4	points per mile
swim_per_100m	1	points per 100 m swum
run_ascent_per_100m	4	points per 100 m climbed running
walk_ascent_per_100m	2	points per 100 m climbed walking
cycle_ascent_per_100m	1	points per 100 m climbed cycling
min_distance_m	100	an activity shorter than this earns nothing

A change re-scores every week, past ones included — lowering a rate lowers what is still owed. A week already marked paid keeps its paid figure. Changes made on the PC reach the page after `git push`; changes made through the form or a reply are live at the next hourly sync.

Argo — on the PC

```
cd C:\Argo
python check.py # every selftest + pyflakes (run after any edit)
python -m argo.sync --dry-run # what Garmin has that we do not, without fetching
python -m argo.sync # fetch it (the Action does this hourly anyway)
python -m argo.score --print # the ledger as a table; rebuilds docs/data.json
python -m argo.statement --print # the last completed week's statement, printed
python -m argo.pay # dry run: the oldest unpaid week (--apply to mark it)
python -m argo.pay --through 2026-09-14 --apply # settle every unpaid week up to that Monday
python -m argo.pay --strike 12345678 --reason "the car" --apply
```

```
python -m argo.pay --sport 12345678 kayak --apply
python -m argo.demo          # a made-up data.json to look at the page design
git add data docs && git commit -m "why" && git push  # PC changes reach the page this way
```

Scripts that write are dry-run by default and take --apply. The design log (why everything is the way it is) is
C:\Argo\ARGO_BRIEF.md; the README has the same instructions as this page.

Atlas — Joe's stats

No money, no rewards. Every hour the Action fetches Joe's activities *and their timelines* (the TCX file of every activity), recomputes everything and publishes the page. Six tabs:

tab	what is on it
Overview	all-time distance / time / climb / streak; this week against last; the last 12 weeks by sport; totals per sport; the latest activities
Records	per sport, the fastest 400 m · 800 m · 1 km · 1 mile · 2 km · 3 km · 2 miles · 5 km · 5 miles · 10 km · 15 km · 10 miles · 20 km · half · 30 km · marathon (runs); 1 km → 100 miles in fifteen steps (rides); walk, swim and kayak ladders; longest, most climb, longest time, fastest average; biggest week and month. A record is the fastest stretch INSIDE an activity — the quickest 5 km within a 12 km run. Tap one for its progression chart.
Rank	every activity by the Atlas score (distance by sport plus climb — a ranking axis, nothing more), overall or within a sport
Progress	weekly and monthly distance stacked by sport; monthly climb; running pace and heart rate over time (fitness is heart rate coming down at the same pace); the same for cycling speed; cumulative distance this year; consistency
Log	every activity in a table — tap a heading to sort, a chip to filter by sport, a row for the detail
Map	every track at once, coloured by sport; tap a line for the activity

Tapping any activity opens its sheet: the numbers, its map, its best efforts with **PB** where it holds the record.

Corrections

The watch is occasionally wrong. Two corrections, both applied before anything is computed:

```
cd C:\Atlas
python -m atlas.fix --strike 12345678 --reason "watch left running" --apply
python -m atlas.fix --unstrike 12345678 --apply
python -m atlas.fix --sport 12345678 kayak --apply          # the watch said "other"
git add data && git commit -m "why" && git push
```

Ids are in the Log tab's detail sheet (the address of the activity on Garmin Connect ends in the same number).

Atlas — on the PC

```
cd C:\Atlas
python check.py                                # every selftest + pyflakes
python -m atlas.sync --dry-run                  # what Garmin has that we do not
python -m atlas.stats --print                   # headline numbers, records, top 10; rebuilds docs/data.json
python -m atlas.demo                           # a made-up data.json to look at the page design
```

The scoring rates behind the rankings are in `C:\Atlas\atlas\scoring.py`; the record ladders in `records.py`. The design log is `ATLAS_BRIEF.md`.

How it all runs, and what to do when it doesn't

when	what	where to look
seven minutes past every hour	sync: Garmin → data → page (both apps). Commits only when something new landed.	Actions → sync
Sunday ~21:45	Argo's statement issue opens (it syncs first, so a Sunday-afternoon ride is on it)	Actions → statement; Issues
when you reply	Argo's paid workflow reads the first line of your comment and acts on it	Actions → paid
when you run it	settings (Argo), pay (Argo)	Actions → the workflow → Run workflow

The page not updating

Open Actions and look at the latest *sync*. Green: the page updates within a minute or two (GitHub's cache holds a file for up to ten minutes; a pull-to-refresh on the phone gets the new one). Red with "authentication" or "login" in

the log: the Garmin tokens have expired — run `python -m argo.login` (or `atlas.login`) again. Red for another reason: the log names the line; send it to me.

A sport the watch names oddly

It shows as *other*, earns nothing (Argo) and is flagged on the statement. Relabel it with `sport <id> kayak`; if the watch keeps using that name, tell me and it is a one-word change so it maps automatically from then on.

Two things to keep in mind

Both pages are public at guessable addresses, with the boys' first names and their routes on them. And the hourly cadence is GitHub's schedule, which is best-effort — a run can be a few minutes late.